

Sistemas Embebidos

Laboratorio 2: Desarrollo de AI y Control Web

Ejercicio 2.2: Modelos

5 de Mayo, 2026

1. Ejercicio 2.2: Modelos

En este ejercicio se entrenaron cinco topologías distintas de redes neuronales para detectar la presencia del identificador y estimar el cuadrante de la imagen donde este se encuentra. Todas las redes fueron entrenadas utilizando imágenes de tamaño 128×128 píxeles. El conjunto de datos contiene imágenes obtenidas desde tres fuentes: fotografías tomadas con celular, fotografías tomadas con ESP en blanco y negro, y fotografías tomadas con ESP en color.

Cada imagen posee dos etiquetas principales. La primera indica la presencia del identificador, donde el valor 0 representa ausencia y el valor 1 representa presencia. La segunda etiqueta indica el cuadrante vertical donde aparece el identificador. La imagen fue dividida en tres zonas verticales iguales, por lo que la etiqueta de cuadrante puede tomar los valores 0, 1, 2 o 3. El valor 0 indica que no hay identificador presente en la imagen.

Las redes fueron diseñadas manteniendo las topologías lo más pequeñas posible, considerando que posteriormente podrían ser utilizadas en una plataforma embebida como ESP32. Se utilizaron arquitecturas de tipo *fully connected* y redes convolucionales. Cada modelo fue exportado a formato `.tflite` cuantizado en `int8`, ya que este formato es más adecuado para despliegue en sistemas embebidos.

1.1. Descripción general de las topologías

Se entrenaron cinco modelos distintos. Las dos primeras redes corresponden a modelos *fully connected*, mientras que las tres restantes corresponden a redes convolucionales. Las redes convolucionales fueron incluidas porque suelen ser más eficientes para procesamiento de imágenes, ya que aprovechan la estructura espacial de los datos.

Cuadro 1: Resumen de las topologías entrenadas.

Modelo	Tipo	Descripción
red_1_mlp_pequena	Fully connected	Red densa pequeña. Aplana la imagen completa y utiliza una capa oculta de 32 neuronas.
red_2_mlp_profunda	Fully connected	Red densa más profunda, con capas ocultas de 128, 64 y 32 neuronas.
red_3_cnn_tiny	CNN	Red convolucional pequeña con dos capas convolucionales y pocas neuronas finales.
red_4_cnn_small	CNN	Red convolucional más grande, con mayor cantidad de filtros y mejor capacidad de aprendizaje.
red_5_cnn_depthwise	CNN depthwise	Red basada en convoluciones separables tipo <i>depthwise</i> , buscando reducir operaciones y tamaño.

1.2. Métricas consideradas

Para comparar los modelos se utilizaron las siguientes métricas:

- **Accuracy de presencia:** mide qué tan bien la red identifica si el objeto está presente o no en la imagen.
- **Accuracy de cuadrante:** mide qué tan bien la red clasifica el cuadrante donde se encuentra el objeto.
- **Número de parámetros:** corresponde a la cantidad de pesos entrenables de la red.
- **Tamaño del modelo:** se reporta el tamaño del modelo en formato TFLite `int8`, ya que este es el más relevante para una implementación embebida.
- **Número de operaciones:** se estimó usando el número de operaciones MAC, es decir, operaciones de multiplicación y acumulación.

En este caso, el modelo tiene dos salidas: una salida binaria para presencia y una salida multiclase para cuadrante. Por esta razón se reportan dos accuracies distintas.

1.3. Resultados cuantitativos

La Tabla 2 muestra los resultados obtenidos para las cinco topologías entrenadas. El tamaño reportado corresponde al archivo `.tflite` cuantizado en `int8`. El número de operaciones corresponde a una estimación de MACs para cada arquitectura.

Cuadro 2: Resultados de accuracy, tamaño y número de operaciones por modelo.

Modelo	Parámetros	Tamaño int8 [bytes]	Tamaño int8 [KB]	MACs	Acc. presencia	Acc. cuadrante
red_1_mlp_pequena	1,573,061	1,577,352	1540.38	1,573,024	89.62 %	53.77 %
red_2_mlp_profunda	6,302,085	6,312,552	6164.60	6,301,856	89.62 %	53.77 %
red_3_cnn_tiny	1,749	7,200	7.03	1,179,984	89.62 %	53.77 %
red_4_cnn_small	17,877	26,776	26.15	14,746,784	90.57 %	59.43 %
red_5_cnn_depthwise	4,357	16,408	16.02	1,375,480	89.62 %	53.77 %

1.4. Análisis de los resultados

A partir de los resultados se observa que las redes *fully connected* poseen una cantidad muy alta de parámetros en comparación con las redes convolucionales. Esto ocurre porque estas redes aplanan la imagen completa de entrada, la cual tiene dimensión $128 \times 128 \times 3$. Al transformar la imagen en un vector, se genera una entrada de 49,152 valores. Por lo tanto, incluso una capa densa pequeña produce una gran cantidad de conexiones.

La red `red_1_mlp_pequena` tiene 1,573,061 parámetros y un tamaño cuantizado de aproximadamente 1.54 MB. La red `red_2_mlp_profunda` es aún más grande, con 6,302,085 parámetros y un tamaño cuantizado cercano a 6.16 MB. Estos tamaños son poco convenientes para una implementación en ESP32, especialmente considerando las restricciones de memoria y almacenamiento del sistema embebido.

Por otro lado, las redes convolucionales presentan tamaños mucho menores. La red `red_3_cnn_tiny` tiene solo 1,749 parámetros y un tamaño cuantizado de 7.03 KB. La red `red_5_cnn_depthwise` tiene 4,357 parámetros y un tamaño de 16.02 KB. Ambas son mucho más adecuadas para un sistema embebido que las redes densas.

La red `red_4_cnn_small` obtuvo el mejor desempeño general, alcanzando un accuracy de presencia de 90.57 % y un accuracy de cuadrante de 59.43 %. Sin embargo, también fue la red con mayor número de operaciones entre las CNN, con aproximadamente 14.75 millones de MACs. Esto indica que, aunque es la red más precisa, también tendría un mayor costo computacional al ejecutarse en la ESP32.

La red `red_3_cnn_tiny` aparece como una alternativa especialmente interesante, ya que mantiene un accuracy de presencia de 89.62 %, usa solo 1,749 parámetros, pesa aproximadamente 7.03 KB en formato int8 y requiere cerca de 1.18 millones de MACs. Aunque su accuracy de cuadrante no supera al de `red_4_cnn_small`, su bajo tamaño y bajo costo computacional la hacen una candidata fuerte para implementación embebida.

1.5. Visualizaciones en Netron

A continuación se presentan las visualizaciones de las cinco redes en Netron. Para estas visualizaciones se utilizaron los modelos `.tflite` cuantizados en `int8`, ya que este formato representa de mejor manera el tipo de modelo que se utilizaría posteriormente en la ESP32.

1.5.1. Red 1: MLP pequeña

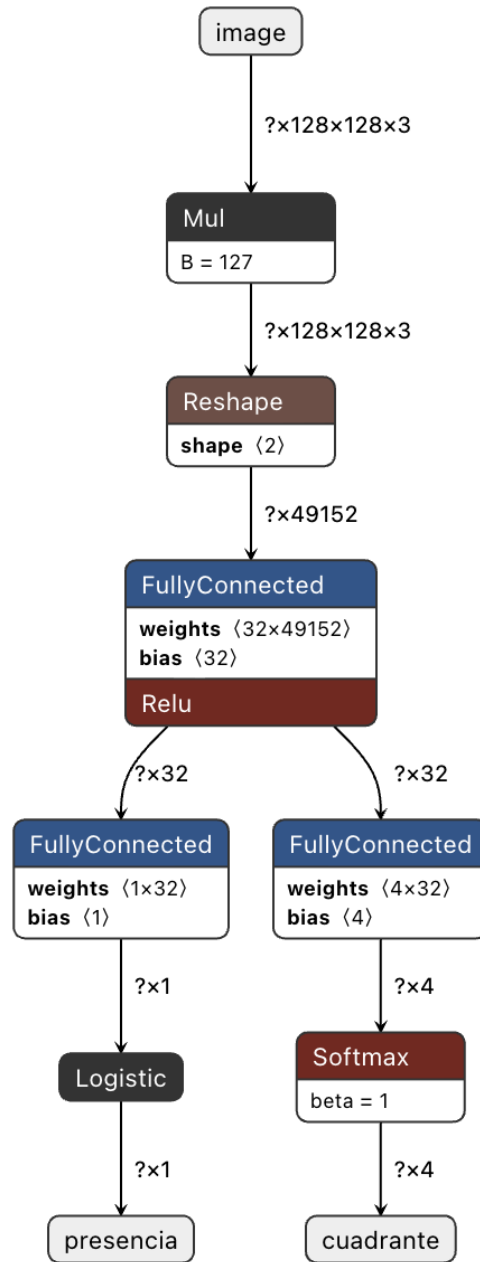


Figura 1: Visualización en Netron de `red_1_mlp_pequena_int8.tflite`.

Esta red corresponde a una arquitectura *fully connected* pequeña. La imagen de entrada se aplanar y luego pasa por una capa densa de 32 neuronas. Finalmente, la red genera dos salidas: una para detectar presencia y otra para clasificar cuadrante. Su principal desventaja es que, al aplanar la imagen completa, genera una cantidad alta de parámetros.

1.5.2. Red 2: MLP profunda

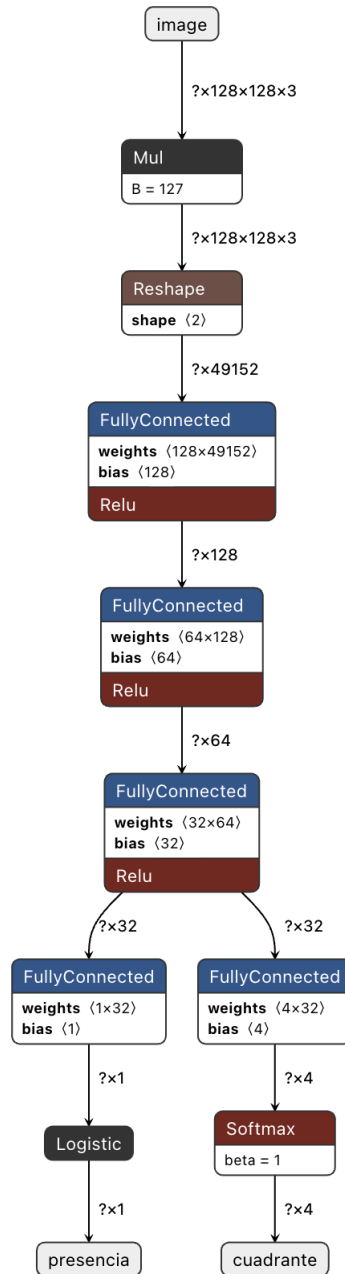


Figura 2: Visualización en Netron de `red_2_mlp_profunda_int8.tflite`.

Esta red también es de tipo *fully connected*, pero posee más capas densas que la red anterior. Esto aumenta su capacidad de representación, pero también incrementa fuertemente la cantidad de parámetros y el tamaño final del modelo. En los resultados obtenidos, no se observa una mejora significativa en accuracy respecto de la MLP pequeña, por lo que su mayor tamaño no se justifica para el caso embebido.

1.5.3. Red 3: CNN tiny

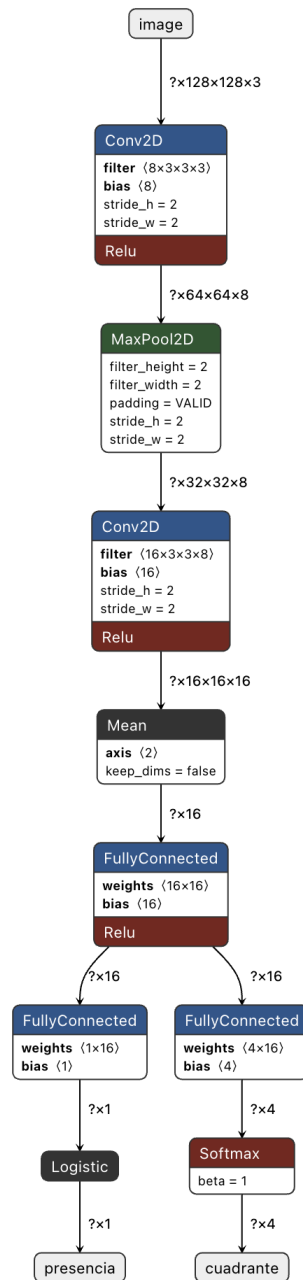


Figura 3: Visualización en Netron de `red_3_cnn_tiny_int8.tflite`.

Esta red utiliza capas convolucionales, por lo que aprovecha la estructura espacial de la imagen. Su tamaño es muy reducido en comparación con las redes densas. Además, mantiene un número bajo de parámetros y un costo computacional moderado. Por estas razones, esta red es una buena candidata para ser implementada en la ESP32.

1.5.4. Red 4: CNN small

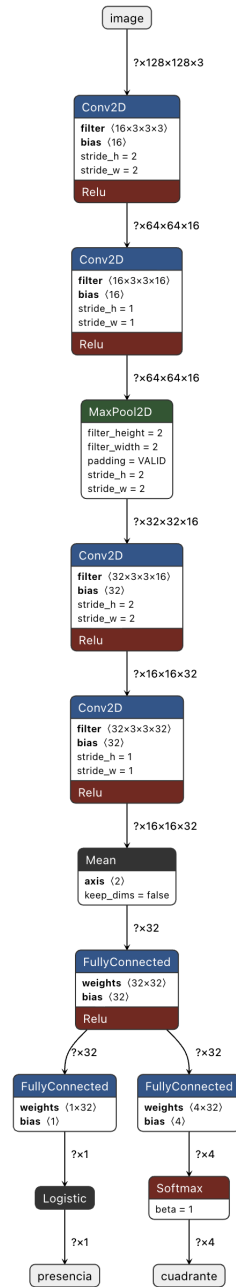


Figura 4: Visualización en Netron de `red_4_cnn_small_int8.tflite`.

Esta red convolucional es más grande que la CNN tiny. Tiene una mayor cantidad de filtros y capas convolucionales, lo que le permite obtener el mejor accuracy de los cinco modelos evaluados. Sin embargo, también presenta el mayor número de operaciones entre las redes convolucionales. Por lo tanto, representa una alternativa más precisa, pero menos eficiente para una ejecución rápida en ESP32.

1.5.5. Red 5: CNN depthwise

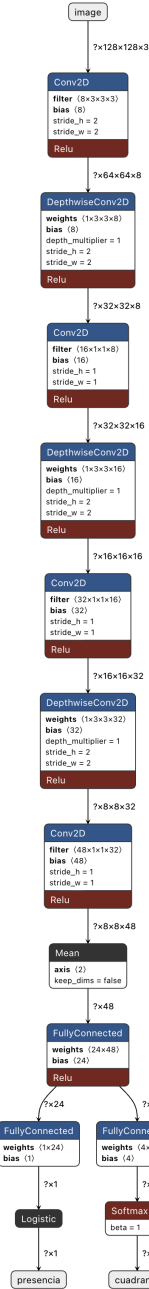


Figura 5: Visualización en Netron de `red_5_cnn_depthwise_int8.tflite`.

Esta red utiliza bloques de convolución *depthwise separable*. Este tipo de arquitectura busca reducir el número de operaciones en comparación con convoluciones tradicionales. En este caso, la red obtiene un tamaño reducido y un número de parámetros bajo. Sin embargo, no logró superar el accuracy de la CNN small. Aun así, es una arquitectura interesante para sistemas embebidos debido a su eficiencia.

1.6. Comparación final de modelos

Al comparar las cinco redes, se observa que las redes densas no son convenientes para este problema en un contexto embebido. Aunque logran un accuracy de presencia similar al de las redes convolucionales, su tamaño es considerablemente mayor. La red `red_2_mlp_profunda`, por ejemplo, alcanza más de 6 MB en formato `int8`, lo que la hace poco práctica para una ESP32.

Entre las redes convolucionales, la `red_4_cnn_small` obtuvo el mejor resultado de accuracy, especialmente en la clasificación de cuadrante. No obstante, este mejor desempeño viene acompañado de un número mucho mayor de operaciones. En cambio, `red_3_cnn_tiny` y `red_5_cnn_depthwise` son mucho más livianas y presentan tamaños adecuados para despliegue embebido.

Considerando solamente el ejercicio 2.2, la red con mejor desempeño en accuracy es `red_4_cnn_small`. Sin embargo, considerando el objetivo final de utilizar el modelo en una ESP32, la red `red_3_cnn_tiny` representa el mejor equilibrio inicial entre tamaño, cantidad de parámetros y accuracy.

1.7. Conclusión

Se entrenaron cinco topologías de redes neuronales para resolver el problema de detección del identificador y clasificación del cuadrante. Las arquitecturas evaluadas incluyeron dos redes *fully connected*, dos redes convolucionales tradicionales y una red convolucional basada en bloques *depth-wise*. Todas fueron exportadas a formato TFLite `int8` para aproximarse al formato esperado en una implementación embebida.

Los resultados muestran que las redes convolucionales son más adecuadas para este problema que las redes densas, ya que logran un rendimiento similar o superior utilizando muchos menos parámetros. La red `red_4_cnn_small` obtuvo el mejor accuracy, con 90.57 % en presencia y 59.43 % en cuadrante. Sin embargo, la red `red_3_cnn_tiny` logró un buen equilibrio entre desempeño y eficiencia, con un tamaño de solo 7.03 KB y 1,749 parámetros.

Por lo tanto, para una implementación futura en ESP32, se considera que `red_3_cnn_tiny` es la alternativa más conveniente si se priorizan tamaño y eficiencia. Si se prioriza exclusivamente accuracy, la mejor alternativa sería `red_4_cnn_small`.